

부록 텐서(배열) 연산

```

# 셋업
import numpy as np
import tensorflow as tf

# shape 확인

a = np.array(1) # 스칼라
print(a.shape, "\n")

b = np.array([1, 2, 3, 4, 5, 6]) # 벡터
print(b.shape, "\n")

c = np.array([[1, 2, 3], [4, 5, 6]]) # 행렬
print(c.shape)

↔ ()

(6,)

(2, 3)

# reshape(shape 변환)

a = np.arange(1, 13)
print(a, "\n")

b = a.reshape(3, 4)
print(b, "\n")

c = b.reshape(-1)
print(c, "\n")

d = b.flatten() # 1차원 배열로 변환
print(d)

↔ [ 1  2  3  4  5  6  7  8  9 10 11 12]

[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]

[ 1  2  3  4  5  6  7  8  9 10 11 12]

[ 1  2  3  4  5  6  7  8  9 10 11 12]

# squeeze(차원 축소)

a = np.array([[1, 2, 3]])
print(a, "\n")

b = np.squeeze(a) # 차원 축소
print(b, "\n")

c = np.expand_dims(b, axis=0) # 차원 추가
print(c, "\n")

d = tf.squeeze(a)
print(d, "\n")

e = tf.expand_dims(d, axis=0)
print(e)

↔ [[1 2 3]]

[1 2 3]

[[1 2 3]]

tf.Tensor([1 2 3], shape=(3,), dtype=int64)

tf.Tensor([[1 2 3]], shape=(1, 3), dtype=int64)

# 텐서-배열 변환

a = tf.constant([[1, 2, 3], [4, 5, 6]])
print(a, "\n")

```

```

b = a.numpy() # 넘파이 배열로 변환
print(b, "\n")

c = tf.convert_to_tensor(b) # 텐서로 변환
print(c)

tf.Tensor(
[[1 2 3]
 [4 5 6]], shape=(2, 3), dtype=int32)

[[1 2 3]
 [4 5 6]]

tf.Tensor(
[[1 2 3]
 [4 5 6]], shape=(2, 3), dtype=int32)

# 인덱싱/슬라이싱

a = np.array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])

print(a)
print(a[5]) # 인덱싱(특정 요소 선택)
print(a[-1])
print(a[:5]) # 슬라이싱(특정 범위의 요소 선택)
print(a[5:], "\n")

b = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])

print(b)
print(b[0])
print(b[0][1])
print(b[0, 1])
print(b[0][1:])

[0 1 2 3 4 5 6 7 8 9]
5
9
[0 1 2 3 4]
[5 6 7 8 9]

[[1 2 3]
 [4 5 6]
 [7 8 9]]
[1 2 3]
2
2
[2 3]

# zeros(/ones(0/1 배열 생성)

a = np.zeros((2, 3)) # 모든 요소가 0인 배열 생성
print(a, "\n")

b = np.ones((2, 3)) # 모든 요소가 1인 배열 생성
print(b, "\n")

c = tf.zeros((2, 3))
print(c, "\n")

d = tf.ones((2, 3))
print(d)

[[0. 0. 0.]
 [0. 0. 0.]]

[[1. 1. 1.]
 [1. 1. 1.]]

tf.Tensor(
[[0. 0. 0.]
 [0. 0. 0.]], shape=(2, 3), dtype=float32)

tf.Tensor(
[[1. 1. 1.]
 [1. 1. 1.]], shape=(2, 3), dtype=float32)

# random(난수 생성)

np.random.seed(77)
a = np.random.randint(1, 10, size=(2, 3)) # 랜덤 정수
print(a, "\n")

```

```

b = np.random.randn(2, 3) # 정규 분포
print(b.round(2), "\n") # np.random.normal(loc=0, scale=1, size=(2, 3))

c = np.random.rand(2, 3) # 균등 분포
print(c.round(2), "\n") # np.random.uniform(low=0, high=1, size=(2, 3))

tf.random.set_seed(77)
d = tf.random.normal(shape=(2, 3), mean=0, stddev=1) # 정규 분포
print(d.numpy().round(2), "\n")

e = tf.random.uniform(shape=(2, 3), minval=1, maxval=10) # 균등 분포
print(e.numpy().round(2))

```

```

[[ 8.  5.]
 [ 6.  9.  1.]]

[[ 0.77 -0.32  0.81]
 [ 1.07  1.1  -0.03]]

[[0.42 0.19 0.76]
 [0.92 0.72 0.97]]

[[-0.38 -2.26  0.65]
 [-0.48  0.39  0.47]]

[[5.19 3.98 7.3 ]
 [6.12 5.1  1.27]]

```

add/subtract/multiply/divide(사칙 연산)

```

a = np.array([1, 2, 3, 4])
print(a, "\n")

b = np.array([5, 6, 7, 8])
print(b, "\n")

c = np.add(a, b) # a + b
print(c, "\n") # tf.add(a, b)

d = np.subtract(a, b) # a - b
print(d, "\n") # tf.subtract(a, b)

e = np.multiply(a, b) # a * b
print(e, "\n") # tf.multiply(a, b)

f = np.divide(a, b) # a / b
print(f.round(2), "\n") # tf.divide(a, b)

```

```

[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 6  8 10 12]
 [-4 -4 -4 -4]
 [ 5 12 21 32]
 [0.2  0.33 0.43 0.5 ]]

```

sum/sqrt/square/power/exp/log(연산 함수)

```

a = np.array([1, 2, 3])
print(a, "\n")

b = np.sum(a) # tf.reduce_sum(a)
print(b, "\n")

c = np.sqrt(a) # 제곱근
print(np.round(c, 2), "\n")

d = np.square(a) # 제곱
print(d, "\n")

e = np.power(a, 3) # 거듭제곱
print(e, "\n")

f = np.exp(a) # 지수
print(np.round(f, 2), "\n")

g = np.log(a) # 자연 로그(ln)
print(g.round(2), "\n")

```

```
h = np.log2(a) # 로그(밑수 2)
print(h.round(2), "\n")

i = np.log10(a) # 상용 로그(밑수 10)
print(i.round(4)) # print(np.round(i, 4))
```

```
↔ [1 2 3]

6

[1.  1.41 1.73]

[1 4 9]

[ 1  8 27]

[ 2.72  7.39 20.09]

[0.  0.69 1.1 ]

[0.  1.  1.58]

[0.  0.301 0.4771]
```

```
# min/max/median/mean/std/var(통계 함수)
```

```
a = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9])
print(a, "\n")
```

```
b = np.min(a) # 최소값
print(b, "\n")
```

```
c = np.max(a) # 최대값
print(c, "\n")
```

```
d = np.median(a) # 중간값
print(d, "\n")
```

```
e = np.mean(a) # 평균
print(e, "\n")
```

```
f = np.std(a) # 표준편차
print(f"{f:.2f} \n")
```

```
g = np.var(a) # 분산
print(f"{g:.2f}")
```

```
↔ [1 2 3 4 5 6 7 8 9]

1

9

5.0

5.0

2.58

6.67
```

```
# argmax(최대값 인덱스)
```

```
a = np.array([[1, 3, 5], [7, 9, 2], [4, 6, 8]])
print(a, "\n")
```

```
b = np.argmax(a, axis=0) # 최대값의 인덱스
print(b, "\n")
```

```
c = np.argmax(a, axis=1)
print(c, "\n")
```

```
d = np.argmax(a, axis=-1)
print(d, "\n")
```

```
e = np.max(a, axis=0) # 최대값
print(e, "\n")
```

```
f = np.max(a, axis=1)
print(f)
```

```
↔ [[1 3 5]
    [7 9 2]]
```

```

[4 6 8]]

[1 1 2]

[2 1 2]

[2 1 2]

[7 9 8]

[5 9 8]

# transpose(전치 행렬)

a = np.array([[1, 2, 3], [4, 5, 6]])
print(a, "\n")

b = a.transpose(1, 0) # a.T
print(b, "\n") # np.transpose(a)

c = tf.transpose(a)
print(c)

↔ [[1 2 3]
    [4 5 6]]

[[1 4]
 [2 5]
 [3 6]]

tf.Tensor(
[[1 4]
 [2 5]
 [3 6]], shape=(3, 2), dtype=int64)

# dot(내적)

a = np.array([1, 2, 3, 4])
print(a, "\n")

b = np.array([5, 6, 7, 8])
print(b, "\n")

c = np.dot(a, b)
print(c, "\n")

d = tf.reduce_sum(a * b) # tf.tensordot(a, b, axes=1)
print(d)

↔ [1 2 3 4]

[5 6 7 8]

70

tf.Tensor(70, shape=(), dtype=int64)

# matmul(행렬곱)

a = np.array([[1, 2], [3, 4]])
print(a, "\n")

b = np.array([[5, 6], [7, 8]])
print(b, "\n")

c = np.matmul(a, b)
print(c, "\n")

d = tf.matmul(a, b)
print(d)

↔ [[1 2]
    [3 4]]

[[5 6]
 [7 8]]

[[19 22]
 [43 50]]

tf.Tensor(
[[19 22]
 [43 50]], shape=(2, 2), dtype=int64)

```

```
# concatenate(배열 결합)

a = np.array([1, 2, 3, 4])
print(a, "\n")

b = np.array([2, 4, 6, 8])
print(b, "\n")

c = np.concatenate((a, b)) # 배열 연결
print(c, "\n")

d = np.stack((a, b)) # 새로운 축으로 배열 쌓음
print(d, "\n")

e = list(zip(a, b)) # 튜플로 묶어 리스트 생성
print(e, "\n")

f = dict(zip(a, b)) # 딕셔너리 생성
print(f)
```

```
↺ [1 2 3 4]

[2 4 6 8]

[1 2 3 4 2 4 6 8]

[[1 2 3 4]
 [2 4 6 8]]

[(1, 2), (2, 4), (3, 6), (4, 8)]

{1: 2, 2: 4, 3: 6, 4: 8}
```

```
# sort(정렬)

a = np.array([1, 3, 5, 2, 4, 6])
print(a, "\n")

b = np.sort(a) # 오름차순 정렬
print(b, "\n")

c = np.sort(a)[::-1] # 내림차순 정렬
print(c)
```

```
↺ [1 3 5 2 4 6]

[1 2 3 4 5 6]

[6 5 4 3 2 1]
```

```
# item(스칼라 값 추출)

a = tf.constant([1, 2, 3])
print(a, "\n")

b = a[1] # 텐서
print(b, "\n")

c = a[1].numpy().item() # 텐서에서 스칼라 값 추출
print(c)
```

```
↺ tf.Tensor([1 2 3], shape=(3,), dtype=int32)

tf.Tensor(2, shape=(), dtype=int32)

2
```

